Working Time 🤓

L2: Nivel de ajuste de un modelo y validación cruzada

Es hora de que pongas en práctica todo lo aprendido. 🤓
Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

   *Utilizando el dataset de flores **Iris** que viene incluido en Scikit-Learn, realiza los siguientes pasos:*

   - ➔ ***Entrena un modelo de clasificación K-Nearest Neighbors (KNN)** para predecir la especie de la flor.*
   - ➔ *Evalúa el desempeño del modelo utilizando el **método de retención (Hold-Out)**, dividiendo el dataset en 70% entrenamiento y 30% prueba.*
   - ➔ *Realiza una evaluación utilizando **Validación Cruzada de K iteraciones (K-Fold)** con k=5.*
   - ➔ *Realiza una evaluación utilizando **Validación Cruzada Aleatoria (Shuffle Split)** con 5 divisiones y prueba del 30%.*
   - ➔ *Realiza una evaluación utilizando **Validación Cruzada Dejando Uno Afuera (LOOCV)**.*
   - ➔ *Compara los resultados de las cuatro técnicas utilizando la métrica de **Exactitud (Accuracy)**.*
   - ➔ *Reflexiona: ¿Cuál técnica fue la más robusta? ¿Qué diferencias observaste?*

2. **¿Dónde se lleva a cabo? 🧑‍💻**

   Visual Studio Code

3. **Tiempo de dedicación ⏳**

   2 horas

4. **Recursos**🛠️

   - *Librería* ***Scikit-Learn***
   - *Dataset* ***Iris*** *incluido en Scikit-Learn*
   - *Documentación de referencia:*
     https://scikit-learn.org/stable/user_guide.html
     https://machinelearningmastery.com/k-fold-cross-validation/

5. **Plus** ➕

   ✅ *Cambiar el modelo por un* ***Árbol de Decisión*** *y comparar los resultados con KNN.*

   ✅ *Graficar las curvas de error para visualizar el impacto del sobreajuste y subajuste.*

6. ⚠️**Condición**

   Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío 👇

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split, KFold, cross_val_score, ShuffleSplit, LeaveOneOut

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

import numpy as np


# Cargar dataset

iris = load_iris()

X, y = iris.data, iris.target


# Modelo

model = KNeighborsClassifier(n_neighbors=3)


# 1. Método de retención

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy_holdout = accuracy_score(y_test, y_pred)

print(f"Exactitud - Hold-Out: {accuracy_holdout:.2f}")


# 2. K-Fold Cross Validation
```

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores_kfold = cross_val_score(model, X, y, cv=kf)
print(f"Exactitud - K-Fold: {np.mean(scores_kfold):.2f}")

# 3. Shuffle Split
ss = ShuffleSplit(n_splits=5, test_size=0.3, random_state=42)
scores_shuffle = cross_val_score(model, X, y, cv=ss)
print(f"Exactitud - Shuffle Split: {np.mean(scores_shuffle):.2f}")

# 4. Leave One Out
loo = LeaveOneOut()
scores_loo = cross_val_score(model, X, y, cv=loo)
print(f"Exactitud - Leave One Out: {np.mean(scores_loo):.2f}")
```

Posible salida:

Exactitud - Hold-Out: 1.00

Exactitud - K-Fold: 0.97

Exactitud - Shuffle Split: 0.97

Exactitud - Leave One Out: 0.97

Reflexión:

Aunque el método Hold-Out puede dar buenos resultados, al depender de una sola partición puede ser menos representativo. Las técnicas de validación cruzada (K-Fold, Shuffle Split, LOOCV) permiten obtener una estimación más robusta y confiable del desempeño del modelo.